Elmo: Recursive Virtual Payment Channels for Bitcoin

Anonymised Submission

***Abstract*—A dominant approach towards the solution of the scalability problem in blockchain systems has been the development of layer 2 protocols and specifically payment channel networks (PCNs) such as the Lightning Network (LN) over Bitcoin. Routing payments over LN requires the coordination of all path intermediaries in a multi-hop round trip that encumbers the layer 2 solution both in terms of responsiveness as well as privacy. The issue is resolved by *virtual channel* protocols that, capitalizing on a suitable off-chain setup operation, enable the two endpoints to engage as if they had a direct payment channel between them. Once the channel is unneeded, it can be optimistically closed in an off-chain fashion.**

Apart from communication efficiency, virtual channel constructions have three natural desiderata. A virtual channel constructor is *recursive* if it can also be applied on pre-existing virtual channels, *variadic* if it can be applied on any number of pre-existing channels and *symmetric* if it encumbers equitably all channel participants both in collaborative and non-collaborative execution paths. We put forth the first Bitcoin-suitable recursive variadic virtual channel construction. Furthermore our virtual channel constructor is symmetric and offers optimal round complexity for payments, optimistic closing and unilateral closing. We express and prove the security of our construction in the universal composition setting, using a novel induction-based proof technique of independent interest. As an additional contribution, we implement a flexible simulation framework for on- and off-chain payments and compare the efficiency of Elmo with previous virtual channel constructors.

***Index Terms*—Payment channels; Blockchain; Security**

1. Introduction

The popularity of blockchain protocols in recent years has stretched their performance exposing a number of scalability considerations. In particular, Bitcoin and related blockchain protocols exhibit very high latency (e.g., Bitcoin has a latency of $\sim$1h [21]) and a very low throughput (e.g., Bitcoin can handle at most 7 transactions per second [10]), both significant shortcomings that jeopardize wider use and adoption and are to a certain extent inherent [10]. To address these considerations a prominent approach is to optimistically handle payments via a *Payment Channel Network* (PCN) (see, e.g., [17] for a survey). Payments over a PCN happen *off-chain*, i.e., without adding any transactions to the underlying blockchain. They only use the blockchain as an arbiter in case of disputes.

The key primitive of PCN protocols is a payment channel. Two parties (the *endpoints*) initiate the channel by locking some funds on-chain and subsequently exchange direct messages to update the state of the channel. Crucially, state updates are not posted on-chain and hence they remain unencumbered by the performance limitations of the underlying blockchain protocol, making them a natural choice for parties that interact often. Multiple overlapping payment channels can be combined and form a PCN.

Before the advent of virtual channels, closing a payment channel involved posting the channel state on-chain. Closing should be efficient, i.e., needing $O(1)$ on-chain transactions, independent of how many payments have occured off-chain. It is also essential that any party can unilaterally close a channel as otherwise a malicious *counterparty* (i.e., the other channel participant) could stop an honest party from accessing their funds. This functionality however raises an crucial design consideration: how to prevent malicious parties from posting old states of the channel. This issue can be addressed with a suitable use of *timelocks*, a native Bitcoin Script instruction that prevents a transaction or a specific script from being processed on-chain prior to a specific time (measured in block height). For instance, diminishing transaction timelocks facilitated the Duplex Micropayment Channels (DMC) [12] at the cost of bounding the overall lifetime of a channel. Using timelocks, the Lightning Network (LN) [24] provided a better solution that enabled channels to stay open for an arbitrary duration: the key idea was to duplicate the state of the channel between the two counterparties, say Alice and Bob, and facilitate a punishment mechanism that can be triggered by Bob whenever Alice posts an old state update and vice-versa. The script timelocking is essential to allow an honest counterparty some time to act.

Interconnecting channels in LN enables endpoints to send funds to each other as long as there is a route of payment channels that connects them. This mechanism has the downside of requiring all parties along the path to be actively involved for each payment. Instead, *virtual payment channels* suggest the more attractive approach of an one-time off-chain initialization to set up a virtual payment channel over the preexisting channels, which can then be used for direct payments with complexity —in the optimistic case— independent of the path length. When the virtual channel outlives its usefulness, it can be closed off-chain if the involved parties cooperate. Initial constructions for virtual channels capitalized on the extended functionality of Ethereum [14], [15], while later work [3] brought them closer to Bitcoin compatibility by leveraging adaptor signatures [2]. We note that Bitcoin imposes very strict design

limitations, as it does not support Turing-complete smart contracts nor covenants [23].

We call the parties of the underlying channels *intermediaries*. A virtual channel constructor can be thought of as an *operator* over the underlying payment channel primitive. We identify three natural desiderata for it.

- Recursive. A recursive virtual channel constructor can operate over channels that themselves could be the result of previous applications of the operator. This allows building virtual channels on top of pre-existing virtual channels, allowing the channel structure to evolve dynamically.
- Variadic. A variadic virtual channel constructor can virtualize any number of input payment channels directly, i.e., without leveraging recursion, contrary to a *binary* constructor. This is important for PCNs since it enables applying the operator to build virtual channels of arbitrary length, without the undue overhead of opening, managing and closing multiple virtual channels only to use the one at the "top" of the recursion.
- Symmetric. A symmetric virtual channel constructor offers setup and closing operations that are symmetric in terms of computation, network and storage cost between the two endpoints or the intermediaries (but not necessarily a mix of both) for the optimistic and pessimistic execution paths. Furthermore, payments by the two endpoints incur the same delay. Importantly, this ensures that no party is worse-off or better-off after an application of the operator in terms of accessing the basic channel functionality.

We note that recursiveness, while identified already as an important design property [15], has been achieved for only for DMC-like fixed lifetime channels [18], but was left as an open question for indefinite lifetime channels on Bitcoin [3]. This is because of the severe limitations imposed by the scripting language of Bitcoin-compatible systems. With respect to the other two properties, observe that successive applications of a recursive binary virtual channel operator to connect distant endpoints will break symmetry, since the sequence of operator applications will impact the participants' functions with respect to the resulting channel. This is of particular concern since most previous virtual channel constructors proposed are binary [15], [3], [18].

The primary motivation for recursive channels is adding flexibility in moving off-chain coins quickly, with minimal interaction, and at a low cost, even when the ledger is consistently congested. Without recursiveness and when facing unresponsive channel parties, one has to first close its channel on-chain in order to then use some of its coins with another party, which is as slow as any on-chain transaction and in case of high congestion prohibitively expensive. To move only part the channel coins to a new channel, a party still has to close the entire original channel, even if the channel parties are collaborative. In contrast, a recursive virtual channel permits using some of its coins with other parties by opening off-chain a new virtual channel on top, keeping the remaining coins in the original channel and without involving the parties of the latter. Importantly, users can decide to open a recursive virtual channel long after having established their underlying one. This flexibility can inspire confidence in virtual channels, prompting users to transfer more coins off-chain and reduce on-chain congestion.

A scenario that needs both the recursive and the variadic properties is as follows (Fig. 1): Initially Alice has a channel with Bob, Bob one with Charlie and Charlie one with Dave. Alice opens a virtual channel with Dave over the 3 channels – this needs the variadic property. She later realises she has to pay Eve a few times, who happens to have a channel with Dave. Alice interacts just with Dave and Eve to move half her coins from her virtual channel with Dave to a new one with Eve – this needs the recursive property.

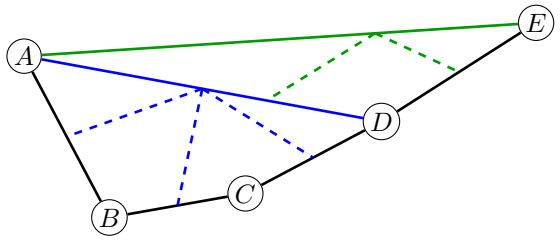

Figure 1: An Elmo network with 5 nodes, 4 black *simple* (i.e., on-chain) channels, 1 blue virtual channel built only on simple ones, and 1 green virtual channel built on the blue virtual and a simple one. Each virtual channel is connected with its *base* channels with dashed lines of the same colour. The variadic and recursive properties are showcased.

Our Contributions. Elmo (named after St. Elmo's fire) is the first Bitcoin-suitable recursive virtual channel constructor that supports channels of indefinite lifetime. In addition, our constructor is variadic and symmetric. Both optimistic and pessimistic execution paths are optimal in terms of round complexity: issuing payments between two endpoints requires just three messages of size independent of the channel length, closing a channel cooperatively needs at most three messages from each party while closing a channel unilaterally demands up to two on-chain transactions for any involved party (endpoint or intermediary) that can be submitted simultaneously, also independent of the channel length. We build Elmo on top of Bitcoin, as this means it can be adapted for any blockchain that supports Turing-complete smart contracts such as Ethereum [26]. The latter provides additional tools to increase Elmo efficiency. Furthermore, Elmo can inspire future blockchain designs that maintain minimal scripting capabilities while providing robust off-chain functionality.

We achieve the above by leveraging a sophisticated virtual channel setup protocol which, on the one hand, enables endpoints to use an interface that is invariant between on-chain and off-chain (i.e., virtual) channels, while on the other, parties can securely close the channel cooperatively off-chain, or instead close unilaterally on-chain, following an arbitrary activation sequence. The latter is achieved by enabling anyone to start closing the channel, while subse-

quent respondents, following the activation sequence, can choose the right action to complete the closure process by posting a single transaction each.

We formally prove the security of our protocol in the Universal Composition (UC) [7] setting (full version [1]); our *global* functionality $\mathcal{G}_{\mathrm{Chan}}$ represents a single channel (on globality, see [6] and Sec. 3). We also guarantee that the real-world protocol Π_{Chan} has *balance security* [16], i.e., that honest parties can close the channel and get the expected coins on-chain (Lemma 1). Elmo requires the `ANYPREVOUT` signature type, a candidate for inclusion in the next Bitcoin update [11]. Today, every signature of a Bitcoin transaction T must sign the hash of all transactions spent by T (more precisely, the spent *outpoints*), thus we currently cannot store a signed transaction T off-chain and decide only later which on-chain transaction(s) it will spend. An `ANYPREVOUT` signature fixes this by not including this hash in the signed message. This enables a transaction signed with an `ANYPREVOUT` signature to spend any output as long as it has a suitable script, even an output that was unknown at the time of signing.

We conjecture that without `ANYPREVOUT` no efficient virtual channel over Bitcoin can be built. In particular, if any protocol (i) has an efficient closing operation with $O(1)$ on-chain transactions, (ii) has parties locally store the channel state as transactions and signatures and (iii) does not need locking up coins as collateral (as [4] does), then each party needs space exponential in the number of intermediaries. The second requirement is natural, since, to our knowledge, all trustless layer 2 protocols over Bitcoin require all implicated protocol parties to actively sign off every state transition and locally store their counterparties' relevant signatures to ensure they can unilaterally exit later.

Related work. Perun [14], Thunderdome [5], and GSCN [15] build virtual state channels on the Turing-complete language of Ethereum. GSCN uses a per-channel functionality and a recursive argument similar to that of our UC-security analysis. However, their security argument is flawed, as they incorrectly argue that every level is subroutine respecting wrt the same environment and subroutines. We believe that, given the versatile Ethereum scripting, GSCN could be straightforwardly extended to support variadic channels. Similar features are provided by Celer [13]. Hydra [8] provides state channels for Cardano [9].

Closer to Elmo, several works propose virtual channel constructions for Bitcoin. LVPC [18] enables a virtual channel to be opened on top of two preexisting channels using a technique similar to DMC, unfortunately inheriting the fixed lifetime limitation. Let *simple channels* be those built directly on-chain, i.e., channels that are not virtual, e.g., LN. Bitcoin-Compatible Virtual Channels [3] enables virtual channels on top of two preexisting simple channels and offers two protocols, the first of which guarantees that the channel will stay off-chain for an agreed period, while the second allows the single intermediary to turn the virtual into a simple channel. This strategy has the shortcoming that even if it is made recursive (a direction left open in [3]) after k applications of the constructor the virtual channel participant will have to publish on-chain k transactions in order to close the channel if all intermediaries actively monitor the blockchain.

Donner [4] (released originally concurrently with the first technical report of Elmo) achieves variadic, but not recursive virtual channels. This is done by having the funder lock as collateral twice the amount of the useful channel funds: once on-chain with funds that are external to the *base channels* (i.e., the channels that the virtual channel is based on) and once off-chain within its base channel. Thus Donner requires the funder to keep locked and unusable an amount of coins equal to those in the virtual channel, creating a considerable opportunity cost. As this is not needed in other constructions, the funder's collateral in Donner is double that of LVPC or Elmo. The collateral for all other parties is the same across LVPC, Donner, and Elmo. Additionally, contrary to Elmo, a Donner channel needs active periodic collaboration of all channel parties to refresh it, thus Donner channels do not have an unlimited lifetime. We conjecture that using external coins precludes variadic virtual channels with unlimited lifetime. This further means that Donner is not symmetric. Donner also uses placeholder outputs which, due to the minimum coins they need to exceed Bitcoin's *dust limit*, may skew the incentives of rational players and add to the channel cost. Its lack of recursiveness mean that if a party with coins in a Donner channel decides to use them with another party, it first has to close its channel either off-chain, which needs cooperation of all intermediaries, or else on-chain, with all the delays and fees this entails. Further, its design complicates future iterations that lift its current restriction that only one of the two channel parties can fund the virtual channel. On the positive side, Donner is more efficient than Elmo in terms of storage, computation and communication, and boasts a simpler design. Their work also introduces *Domino*, which we discuss in [1]. We briefly mention here that, contrary to what is claimed in [4], Domino does not force the base channels of an Elmo virtual channel to close.

We refer the reader to to [1] for further related work, including a technical issue in Donner and its resolution. Table 1 contains a comparison of the features and limitations of virtual channel protocols, including the current work. Future work can be found in [1].

2. Protocol Description

Conceptually, Elmo is split into four actions: channel opening, payments, cooperative closing and unilateral closing. Parties P_1 and P_n may **open** a channel (P_1, P_n) between them directly on-chain, in which case they follow an opening procedure like that of LN; such a channel is called *simple*. Otherwise they can open it on top of a path of preexisting *base* channels (P_1, P_2), (P_2, P_3), $\ldots$, (P_{n-1}, P_n). Then (P_1, P_n) is a *virtual* channel. A channel is either simple or virtual, not both. Elmo is recursive, thus each base channel may itself be simple or virtual. To open a virtual channel, all parties on the path set aside funds in their

Table 1: Features & requirements comparison of virtual channel protocols

	Unlimited lifetime	Recursive	Variadic	Symmetric	Script requirements
LVPC [18]	✗	◐[a]	✗	✓	Bitcoin
BCVC [3]	✓	✗	✗	✓	Bitcoin
Perun [14]	✓	✗	✗	✓	Ethereum
GSCN [15]	✓	✓	✗	✓	Ethereum
Donner [4]	✗	✗	✓	✗	Bitcoin
this work	✓	✓	✓	✓	Bitcoin + `ANYPREVOUT`

a. lacks security analysis

channels as collateral; to do this they create $O(n^3)$ so-called *virtual* and *bridge* transactions (txs) in total ($O(n^2)$ stored per party) that essentially tie the spending of two adjacent base channels into a single atomic action. These txs are kept off-chain. The bridge txs have a *funding* output with the sum of P_1 and P_n's channel balance. To complete channel opening, P_1 and P_n then apply the logic of simple channels (see below) on top of the (off-chain) funding output. LN demands that the funding output is on-chain, but we lift this requirement. We instead guarantee that either endpoint can put the funding output on-chain unilaterally.

In more detail, the channel opening protocol steps are as follows: the two counterparties (i) create new signing keypairs and exchange the resulting public keys, then (ii) if the channel is virtual, they prepare the underlying base channels, next (iii) they exchange signatures for their initial *commitment* txs (see Simple channel below) and lastly, (iv) if the channel is simple, the *funder* (who is the only party that provides coins for a new channel, like in LN) signs and publishes the *funding* tx to the ledger. A **payment** over an established channel (see Simple channel below) follows a procedure inspired by LN.

Parties $P_1, \ldots, P_n$ can **optimistically close** a virtual channel fully off-chain. At a high level, parties controlling base channels *revoke* their *virtual* txs and the commitment txs, i.e., they cannot use them anymore. This effectively "peels" one *virtualisation layer*. In the process, they redistribute coins so that intermediaries "break even", while P_1 and P_n get their rightful coins (as per the last channel state) in their base channels ((P_1, P_2) and (P_{n-1}, P_n) respectively).

Finally, to **unilaterally close** a virtual channel (P_1, P_n) in case cooperation fails, either P_1 or P_n sign and publish some txs on-chain. In the simplest case, P_1 publishes her virtual tx. This prompts P_2 to publish her relevant virtual tx and so on up to P_{n-1}. The funding output of (P_1, P_n) is then on-chain and closing can proceed as for simple channels. If instead any intermediary stays inactive, then a timelock expires turning an output into the funding output for (P_1, P_n), at the expense of the inactive party. As discussed later, the funding output employs `ANYPREVOUT` so that the channel needs only a single commitment tx per endpoint, avoiding an exponential state blowup in the recursion depth and making off-chain payments efficient. We assume that parties already have established secure communication channels. In practice, peer discovery can be done as in LN.

In order to build intuition, let us present examples of the lifecycles of a simple and a virtual channel. Consider 5 parties, $A, B, \ldots, E$ and 4 channels, $(A, B), \ldots, (D, E)$, that will act as the base of the virtual channel (A, E). We first follow the operations of the simple channel (A, B) and then those of (A, E). We simplify some parts of the protocol to aid comprehension — more details are in [1].

Simple channel. First A and B generate funding and revocation keypairs and exchange public keys (A_F, B_F, A_R, B_R in Fig. 2 – 2 messages). Each then locally generates the *funding* and the two *commitment* txs (F, C_A in Fig. 2). They sign C_A and exchange the signatures (2 messages). A stores her signed commitment tx off-chain and publishes F on-chain. Once F is finalised, the channel is *open*.

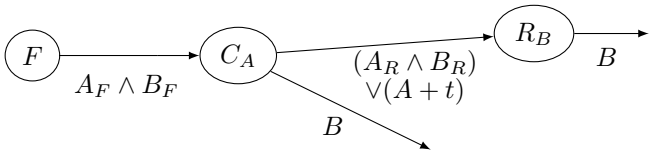

Figure 2: A–B simple channel: Funding (F), A's commitment (C_A) and B's revocation (R_B) txs. $A_F \wedge B_F$ is the funding output. The (symmetric) B's commitment and A's revocation txs are omitted. A_F is A's funding key, A_R her revocation key, $A \wedge B$ needs a signature by both A and B. $A + t$ needs a signature by A after relative timelock t. The top C_A output is spendable either by both A_R and B_R, or by $A + t$, the "or" is denoted with $\vee$.

The funding tx F moves A's initial coins to a 2-*of*-2 *multisig*, i.e., an output that needs signatures from A_F and B_F to be spent. There is one commitment tx per party, stored locally off-chain. The one held by A (C_A, Fig. 2) spends F and has one output for A (initially with all coins) and one for B (initially with 0 coins). A's output can be spent by either a multisig, or by A after a *relative timelock* of t (relative means the countdown starts at the moment of publication). This is so that B has time to *punish* A if she cheats. B's commitment tx C_B is symmetric. We recommend using $t = 5$ days as a default, the same as LN.

When A *pays* c coins to B, A and B create two new commitment txs. They have the same outputs and scripts as their previous ones, save for the coins: A's outputs have c coins less, B's have c more. They sign them and swap signatures. To ensure only one set of commitment txs is valid, they then revoke their previous ones by building and

signing *revocation* txs for the previous commitment txs. B's revocation tx (R_B, Fig. 2) gives B all coins that belonged to A and vice versa. This disincentivises both parties from publishing an old commitment tx under the threat of losing all their coins. A payment needs 3 messages in total. Note that, after generating the new commitment txs and before revoking the old ones, each party has two valid commitments for different channel versions. This is fine, as the payment only concludes when revocation completes. If one party publishes the new commitment tx before revocation, then the other will consider the payment successful and vice versa for the old commitment tx.

To unilaterally *close* (A, B), A or B publish the latest commitment tx on-chain and wait for timelock expiration. The last commitment tx is not revoked, thus punishment is impossible. Simple Elmo channels work like LN simplifed.

Virtual channel. Let channels $(A, B), \ldots, (D, E)$ be open and each "left" party own at least c coins. Let us see how to open the virtual channel (A, E) with A initially having c coins using $(A, B), \ldots, (D, E)$ as base channels. Each base channel may be simple or virtual. If any one is virtual, then (A, E) is recursive. Thanks to the similarity of all layers, the description below is identical in both cases. First, the 5 parties generate and exchange a number of new keys, i.e., all keys that appear in the outputs of Figs. 3–6. Then each base pair removes c coins from the "left" party in their base channel. The updated commitment txs use some of the new keys for the multisig in their input, since, as we will see, so-called *virtual* txs will stand between the funding and the commitment txs from now on. These virtual txs will form the *virtual layer* (Fig. 7).

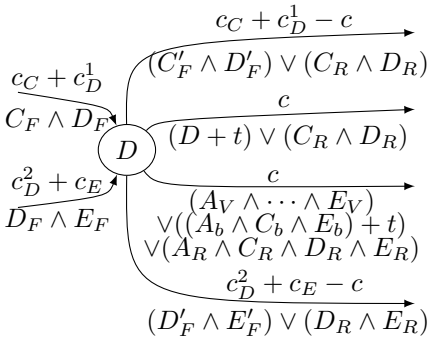

Figure 3: A–E virtual channel: D's initiator virtual tx. Spends the funding outputs of the C–D and D–E channels. D can use it if neither C nor E have published a virtual tx yet. c_D^1/c_D^2: D's coins in C–D/D–E channel. A_V: A's *virtual* key. A_b: A's *bridge* key.

Next, parties generate and sign these virtual txs and exchange the signatures. These txs form Elmo's core. Intuitively, they force each intermediary to interact with both of its base channels instead of one at a time, so that virtual channel information flows along the base channels. There are 3 virtual tx types – looking ahead, we explain them based on how they help close the channel on-chain. The

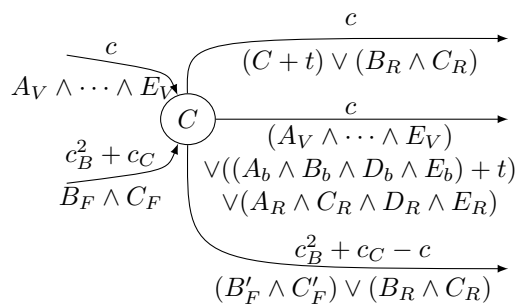

Figure 4: A–E virtual channel: One of C's virtual extend-interval txs. Spends the virtual output of D's initiator tx and the funding output of the B–C channel. C can use it if D has already published its initiator tx and B has not published a virtual tx yet.

initiator virtual tx spends the two funding outputs of an intermediary (say D) and needs to be signed by C_F, D_F, and E_F. It has 4 outputs: one new funding output for each of (C, D) and (D, E) (Fig. 3, top & bottom outputs), one output that refunds the collateral to D (2nd from top) and, crucially, a so-called *virtual* output (3rd from top). The latter can be spent by C by the top input of an *extend-interval* tx, the second type of virtual tx (Fig. 4). It needs signatures from the *virtual keys* $A_V \ldots E_V$ of all parties. This tx also spends the other C's funding output, namely that of (B, C) (bottom input). It has 3 outputs: one refunding the collateral to C (top), another virtual output (middle), and a funding output that replaces the one just spent (bottom). In our example, A also uses its initiator tx (Fig. 5). The format of an endpoint's initiator tx is different. It spends only the funding output of (A, B) and produces 2 outputs: a new funding output for (A, B) (top) and a virtual output (bottom) – this is the only virtual tx A stores. B can in turn spend A's and C's virtual outputs with a *merge-intervals* tx (Fig. 6), the last virtual tx type, which also needs signatures from $A_V \ldots E_V$. It has 2 outputs: one refunds B and the other produces a new virtual output. Now all base funding outputs are spent, all intermediaries are refunded and the virtual layer has "collapsed". Additionally, the virtual output of B's tx plays the role of the funding output of the virtual channel (A, E).

The virtual txs have two properties: First, each intermediary can publish at most one virtual tx, which refunds its collateral exactly once. We will see how this is enforced below. Second, if the chain of virtual txs is at any point broken by an inactive intermediary that does not publish its virtual tx, the virtual channel will still be funded correctly and the inactive party will lose its collateral. This is because the unclaimed virtual output turns into the funding output of the virtual channel after a timelock, see, e.g., Fig. 3, output 3, spending method 2.

Keys $A_b \ldots E_b$ (Figs. 3–6) can be spent by *bridge* txs, the output of which funds (A, E). Bridge txs allows endpoints to only store a single commitment tx instead of

$O(n^3)$. Any virtual output may end up funding the virtual channel, but each virtual output has a different script, thus no single commitment tx is able to spend all of them. Bridge txs fix this by having the same output. This makes virtual channel updates as cheap as simple channel updates.

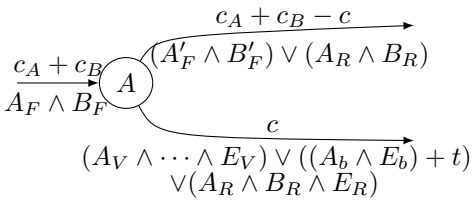

Figure 5: A–E virtual channel: A's virtual initiator tx. Spends the funding output of the A–B channel. A can use it if B has not published a virtual tx.

Some considerations remain to ascertain the scheme's security. Firstly, if all virtual outputs were identical, an adversary could fabricate arbitrarily many virtual outputs using its own, external to the protocol, coins and publish a perpetual stream of merge-intervals txs that spend one valid and one fabricated virtual output. This would prevent an honest endpoint from accessing their funding output. We avoid this by specifying on each virtual output the exact sequence of parties that have already published a virtual tx and only allowing the parties at the two edges of the sequence to extend it with their virtual tx. If all intermediaries publish a virtual tx, then the last published one cannot be spent by another virtual tx. This ensures that the endpoints will eventually obtain a funding output, but needs intermediaries to store $O(n^2)$ virtual txs for a virtual channel over n parties. Secondly, we have to carefully select the values of timelocks to ensure that each party has enough time to act. Timelocks increase linearly with the recursion depth. The exact values are shown in [1].

We now return to the opening procedure. After the 5 parties set up the virtual txs, they revoke their previous commitment txs. They do this by signing the relevant revocation txs, just like for a simple channel. It takes $12 \cdot (n-1)$ messages, i.e., 6 messages per endpoint and 12 messages per intermediary, to set up the virtual layer — see Sec. 4 for more details.

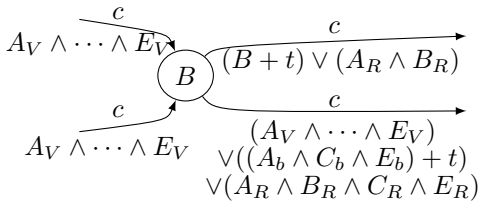

Figure 6: A–E virtual channel: One of B's merge-intervals txs. Spends the virtual outputs of A's and C's virtual txs. B can use it if both A and C have already published their initiator or extend-interval tx.

At last, the virtual layer is set up: Both A and E can unilaterally force a funding output for channel (A, E) on-chain, no matter what the rest of the parties do. Likewise, honest intermediaries can unilaterally retrieve their funds.

A and E finally exchange commitment transactions for their new channel, thus concluding its opening. A and E can pay each other over their virtual channel exactly like they would over a simple channel; we refer the reader to the relevant description above.

Note that funding outputs use the `ANYPREVOUT` flag, thus ensuring that a single pair of commitment txs can spend any of the funding outputs. If `ANYPREVOUT` were not used, each virtual layer would need a copy of the entire set of discussed txs for each possible funding output of its base layer, resulting in exponential storage requirements. To make matters worse, a payment over (A, E) would need A and E to renegotiate exponentially many commitment txs, as well as recalculate all their downstream txs, which would in turn need interaction with intermediaries of all virtual channels built over (A, E), completely defeating the essence of payment channels.

Cooperative closing. To enhance usability, our protocol enables closing the virtual channel off-chain if all parties cooperate. To do this, the endpoints first let the intermediaries know their final virtual channel balance. Then the parties of each base channel create new commitment txs for their channels, moving the collateral back into the channel: the "left" party gets A's coins and the "right" one gets E's. Thus all intermediaries "break even" across their two channels. Once they do this, the parties revoke all virtual txs, using a logic similar to the revocation procedure of simple channels but scaled up to all parties. This is why all virtual tx outputs (Figs. 3–6) have a spending method with $A_R \ldots E_R$ keys.

What if one party does not cooperate? Then one or more of the other parties must close unilaterally on-chain. Fig. 7 shows how this would play out if A and D initiated this procedure.

Our protocol is recursive because both simple and virtual channels are ultimately represented by a funding output that either already is or can be put on-chain, therefore new virtual channels can be built on either. Both simple and virtual channels avoid key reuse on-chain, thus ensuring party privacy from on-chain observers. More details are in [1]

3. Security

Before providing the UC-based security guarantees, it is useful to obtain concrete properties directly from our protocol, essentially providing standard property-based security in addition to our UC guarantees. We first delineate the security guarantees Elmo provides by proving Lemma 1 which discusses the conservation of funds. The formal statements along with all proofs are deferred to [1]. Informally, it establishes that if an honest was implicated in a channel that has now been unilaterally closed, then the party will have at least the expected funds on-chain.

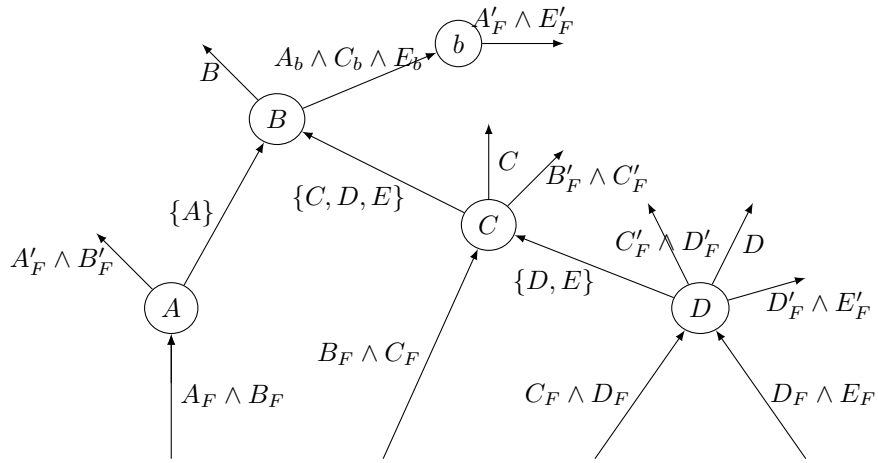

Figure 7: Example of on-chain (unilateral) closing of virtual channel (A, E), built on top of simple channels $(A, B), \ldots, (D, E)$. A and D start closing by publishing their initiator virtual txs (Fig. 5 and 3 respectively), then C publishes its suitable extend-interval virtual tx (Fig. 4), after which B publishes its suitable merge-intervals virtual tx (Fig. 6). No party stays inactive. The virtual transactions A–D form the virtual layer. Virtual outputs are marked with the set (interval) of parties that have already published a tx. *Bridge* txs such as b are used by A and E to convert the various virtual outputs into the same funding output $A'_F \wedge E'_F$, as `ANYPREVOUT` only works across identical outputs.

Lemma 1 (Real world balance security (informal)). Consider a real world execution with $P \in \{Alice, Bob\}$ an honest, non-negligent party. Assume that all of the following are true:

- P opened the channel, with initial balance c,
- P is the funder of n channels, each funded with f_i coins,
- P has cooperatively closed k channels, where the i-th channel transferred r_i coins from the funded virtual channel to P,
- P has sent m payments, each involving d_i coins,
- P has received l payments, each involving e_i coins.

If P performs a successful unilateral close, eventually there will be $h \in \mathbb{N}^*$ outputs on-chain spendable only by P, each of value c_i, such that

$$\sum_{i=1}^{h} c_i \geq c - \sum_{i=1}^{n} f_i + \sum_{i=1}^{k} r_i - \sum_{i=1}^{m} d_i + \sum_{i=1}^{l} e_i \ . \quad (1)$$

The expected funds are [initial balance - funds for funded virtuals + funds returned from funded virtuals - outbound payments + inbound payments]. A similar property-based guarantee is provided for the ideal-world functionality $\mathcal{G}_{\text{Chan}}$ in in [1].

Proof Sketch [Lemma 1]: All execution paths are followed, keeping track of the resulting balance in each case and concluding that balance is secure in all cases, except if signatures are forged. □

It is important to note that in fact our protocol Π_{Chan} provides a stronger guarantee: a party can always successfully unilaterally close its channel and obtain the expected funds on-chain within a known number of blocks. This stronger guarantee is sufficient to make Elmo reliable enough for real-world applications. However an ideal world functionality with such guarantees would have to be aware of specific txs and signatures, making it as complicated as the protocol, thus violating the spirit of the simulation-based security paradigm.

$\mathcal{G}_{\text{Chan}}$ [1] halts on security breaches (e.g., lower than expected balance). We prove the "no-halt" Lemma which informally states that if an ideal party is honest, $\mathcal{G}_{\text{Chan}}$ does not halt with overwhelming probability.

4. Efficiency Evaluation & Simulations

We offer here a cost and efficiency comparison of this work with LVPC [18] and Donner [4]. We focus on these due to their exclusive support of virtual channels over any number of base channels. We remind that LVPC achieves this via recursion, while Donner because it is variadic (cf. Table 1).

We count the communication, storage and on-chain cost of a virtual channel in each protocol. We also simulate the execution of a large number of payments among many parties and derive payment latency and fees – We thus obtain an end-to-end understanding of both the requirements and benefits of each protocol.

Cost calculation. Consider the setting of 1 funder (P_1), 1 fundee (P_n) and $n-2$ intermediaries ($P_2, \ldots, P_{n-1}$) where P_i has a base channel with each of P_{i-1}, P_{i+1}. We compare the costs of off-chain opening (Table 2) and on-chain closing unilaterally (Table 3). Comparison details and choices are deferred to [1].

Regarding opening, in Table 2 we calculate for each of the 3 protocols the number of communication rounds required, the total size of outgoing messages as well as the amount of space for storing channel data. We calculate

Open											
	Funder			Fundee			Intermediary			Total	
	party rounds	size		party rounds	size		party rounds	size		size	
		sent	stored		sent	stored		sent	stored	sent	stored
LVPC	$8(n-2)$	$1381(n-2)$	$3005(n-2)$	7	1254	2936	16	2989	6385	$4370n-8740$	$9390n-18780$
Donner	2	$184n+829$	$1332.5k+$ $43n+125.5$	1	$43n+192.5$	$1332.5k+$ $43n+125.5$	1	547	$1332.5k+$ $43n+125.5$	$774n-71$	$1332.5kn+$ $43n^2+125.5n$
Elmo	6	$32n^3-128n^2$ $+544n-276$	$\frac{128}{3}n^3-128n^2$ $+\frac{1276}{3}n+220$	6	$32n^3-128n^2$ $+544n-340$	$\frac{128}{3}n^3-128n^2$ $+\frac{1276}{3}n+220$	12	$96n^3-256n^2$ $+404n-40$	$96n^3-256n^2$ $+468n+88$	$96n^4-384n^3+$ $724n^2+240n-792$	$96n^4-\frac{1088}{3}n^3+$ $660n^2+\frac{8}{3}n+520$

Table 2: Open efficiency comparison of virtual channel protocols with n parties and k payments.

Unilateral Close								
	Intermediary		Funder		Fundee		Total	
	#txs	size	#txs	size	#txs	size	#txs	size
LVPC	3	627	2	383	2	359	$2n-2$	$435n-510.5$
Donner	1	204.5	4	$704+43n$	1	136.5	$2n$	$458n-26$
Elmo	1	297.5	3	376	3	376	$n+1$	$254.5n-133$

Table 3: On-chain worst-case closing efficiency comparison of virtual channel protocols with n parties.

Delays (sec) and Relative Fees						
	Power Law		Preferred Receiver		Uniform	
	Delay (sec)	Fees	Delay (sec)	Fees	Delay (sec)	Fees
LVPC	2873	0.81	2440	0.80	3589	0.99
Donner	2842	1	2361	1	3589	1
Elmo	2864	0.83	2344	.87	3589	0.99

Table 4: Average per-payment delay (both on- and off-chain) and relative fee. Less is better. Blue is best, red is worst.

funder, fundee and intermediary requirements, along with the aggregate for all parties.

Regarding closing, in Table 3 we calculate for each of the three protocols the worst-case on-chain cost for a party in order to unilaterally close its channel. The cost is measured both in the number of transactions and in their total size.

For the two endpoints (funder and fundee), we show the cost of unilaterally closing the virtual channel, whereas for each intermediary we report the cost of closing a base channel. We also present the worst-case total on-chain cost, aggregated over all parties. Note that the latter cost is not simply the sum of the worst-case costs of all parties, as one party's worst case is not necessarily the worst case of another. This cost rather represents the maximum possible load an instantiation of each protocol could add to the blockchain when closing.

We note that Elmo exploits MuSig2 [19], [22] to reduce both its on-chain and storage footprint: the n signatures that are needed to spend each virtual and bridge output can be securely reduced to a single aggregate signature. The same cannot be said for Donner, since this technique cannot optimise away the n outputs of the funder's transaction $\texttt{tx}^{\texttt{vc}}$. Likewise LVPC cannot gain a linear improvement with this optimisation, since each of its relevant transactions ("split", "merge" and "refund") needs constant signatures.

We furthermore note that, since human connections form a small world [20], we expect that in practice the need for virtual channels with a large number of intermediaries will be exceedingly rare. This is corroborated by the fact that LN only supports payments of up to 20 hops without impact to its usefulness. Therefore, the asymptotic network and storage complexity are not as relevant as the concrete costs for specific, low values of n. Under this light, the overhead of Elmo is tolerable. For example, the total size of messages sent and received for a funder when opening an Elmo channel of length $n = 6$ are less than 3 times those of Donner and slightly cheaper than LVPC (Table 2). Moreover, Elmo can be deployed on commodity hardware: To set up and keep a channel of length $n = 6$ active, an intermediary needs to send less than 14 KiB over the network and store less than 15 KiB — the endpoints need even less resources.

Payment simulations. We implemented a simulation framework[1] in which a list of randomly generated payments are carried out. A simulation run is parametrised by a list of payments (sender, receiver, value triples), the protocol (Elmo, Donner, LVPC, LN or on-chain only), which future payments each payer knows and the utility function it maximises. Our framework is of independent interest, as it can be used for evaluating various payment networks.

We here show the performance of the 3 protocols with respect to the metrics payment channels aim to improve, namely payment latency and fees (Table 4). We have designed 3 workloads: "power law", in which incoming payments follow a zipf [25] distribution, "preferred receiver", in which each party has a preferred payee which receives half of the payments, and "uniform", where payments are chosen uniformly at random. More details are deferred to [1].

As evidenced, Elmo is the best or on par with the best protocol in every case. We attribute this to the flexibility of Elmo, as it is both variadic and recursive, thus able to exploit the cheapest payment method in all scenarios. In particular, Donner is consistently the most fee-heavy protocol and LVPC the slowest. Elmo experiences similar delays to Donner and slightly higher fees than LVPC.

1. `gitlab.com/anonymised-submission-8778e084/virtual-channels-simulation`

References

[1] Anonymous, "Elmo: Recursive virtual payment channels for bitcoin."

[2] L. Aumayr, O. Ersoy, A. Erwig, S. Faust, K. Hostáková, M. Maffei, P. Moreno-Sanchez, and S. Riahi, "Generalized bitcoin-compatible channels," *IACR Cryptol. ePrint Arch.*, p. 476, 2020.

[3] L. Aumayr, M. Maffei, O. Ersoy, A. Erwig, S. Faust, S. Riahi, K. Hostáková, and P. Moreno-Sanchez, "Bitcoin-compatible virtual channels," in *2021 IEEE Symposium on Security and Privacy (SP)*, 2021, pp. 901–918.

[4] L. Aumayr, P. Moreno-Sanchez, A. Kate, and M. Maffei, "Breaking and fixing virtual channels: Domino attack and donner," in *30th Annual Network and Distributed System Security Symposium, NDSS 2023, San Diego, California, USA, February 27 - March 3, 2023*. The Internet Society, 2023.

[5] Z. Avarikioti, Y. Wang, and Y. Wang, "Thunderdome: Timelock-free rationally-secure virtual channels," 2025.

[6] C. Badertscher, R. Canetti, J. Hesse, B. Tackmann, and V. Zikas, "Universal composition with global subroutines: Capturing global setup within plain UC," in *Theory of Cryptography - 18th International Conference, TCC 2020, Durham, NC, USA, November 16-19, 2020, Proceedings, Part III*, 2020, pp. 1–30.

[7] R. Canetti, "Universally composable security: A new paradigm for cryptographic protocols," in *42nd Annual Symposium on Foundations of Computer Science, FOCS 2001, 14-17 October 2001, Las Vegas, Nevada, USA*, 2001, pp. 136–145.

[8] M. M. T. Chakravarty, S. Coretti, M. Fitzi, P. Gazi, P. Kant, A. Kiayias, and A. Russell, "Hydra: Fast isomorphic state channels," Cryptology ePrint Archive, 2020/299.

[9] M. M. T. Chakravarty, R. Kireev, K. MacKenzie, V. McHale, J. Müller, A. Nemish, C. Nester, M. Peyton Jones, S. Thompson, R. Valentine, and P. Wadler.

[10] K. Croman, C. Decker, I. Eyal, A. E. Gencer, A. Juels, A. Kosba, A. Miller, P. Saxena, E. Shi, E. G. Sirer *et al.*, "On scaling decentralized blockchains," in *Financial Cryptography and Data Security*. Springer, 2016, pp. 106–125.

[11] C. Decker and A. Towns, "Sighash_anyprevout for taproot scripts."

[12] C. Decker and R. Wattenhofer, "A fast and scalable payment network with bitcoin duplex micropayment channels," in *Symposium on Self-Stabilizing Systems*. Springer, 2015, pp. 3–18.

[13] M. Dong, Q. Liang, X. Li, and J. Liu, "Celer network: Bring internet scale to every blockchain," 2018.

[14] S. Dziembowski, L. Eckey, S. Faust, and D. Malinowski, "Perun: Virtual payment hubs over cryptocurrencies," in *IEEE Symposium on Security and Privacy (SP)*. Los Alamitos, CA, USA: IEEE Computer Society, May 2019, pp. 344–361.

[15] S. Dziembowski, S. Faust, and K. Hostáková.

[16] G. Fabiański, R. Stefański, and O. S. Thyfronitis Litos, "A formally verified lightning network," in *Financial Cryptography and Data Security*. Cham: Springer International Publishing, 2025.

[17] L. Gudgeon, P. Moreno-Sanchez, S. Roos, P. McCorry, and A. Gervais.

[18] M. Jourenko, M. Larangeira, and K. Tanaka, "Lightweight virtual payment channels," in *Cryptology and Network Security*, 2020, pp. 365–384.

[19] G. Maxwell, A. Poelstra, Y. Seurin, and P. Wuille, "Simple schnorr multi-signatures with applications to bitcoin," *Des. Codes Cryptogr.*, vol. 87, no. 9, pp. 2139–2164, 2019.

[20] S. Milgram, "The small world problem," *Psychology Today*, vol. 1, pp. 61–67, May 1967.

[21] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008.

[22] J. Nick, T. Ruffing, and Y. Seurin, "Musig2: Simple two-round schnorr multi-signatures," in *Advances in Cryptology - CRYPTO 2021 - 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16-20, 2021, Proceedings, Part I*, ser. Lecture Notes in Computer Science, T. Malkin and C. Peikert, Eds., vol. 12825. Springer, 2021, pp. 189–221.

[23] R. O'Connor and M. Piekarska, "Enhancing bitcoin transactions with covenants," in *Financial Cryptography and Data Security*, M. Brenner, K. Rohloff, J. Bonneau, A. Miller, P. Y. Ryan, V. Teague, A. Bracciali, M. Sala, F. Pintore, and M. Jakobsson, Eds. Cham: Springer International Publishing, 2017, pp. 191–198.

[24] J. Poon and T. Dryja.

[25] D. M. W. Powers, "Applications and explanations of Zipf's law," in *New Methods in Language Processing and Computational Natural Language Learning*, 1998.

[26] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger."